

Extending Conditionally Borrowed

Document #: P3117R0
Date: 2024-02-05
Project: Programming Language C++
Audience: SG-9
LEWG
Reply-to: Zach Laine
<whatwasthataddress@gmail.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

- 1 Changelog
- 2 Motivation
- 3 Why care about `borrowed_range`?
- 4 Status quo
 - 4.1 The easy ones
 - 4.2 The low-cost ones
 - 4.3 The other one
- 5 Proposed changes
 - 5.1 `join_view`
 - 5.2 Create implementation-only traits
 - 5.3 The easy ones
 - 5.4 `filter_view`
 - 5.5 `chunk_by_view`
 - 5.6 `lazy_split_view`, `split_view`, and `join_with_view`
- 6 Implementation experience
- 7 Costs versus benefits
 - 7.1 The other easy ones
 - 7.2 The low-cost ones
 - 7.2.1 The multi-view low-cost views
 - 7.3 Costs/benefits conclusion
- 8 References

§ 1 Changelog

Initial revision.

§ 2 Motivation

In [P2017R1], we made some range adaptors conditionally borrowed. But we didn't touch adaptors that had callables - like `views::transform`. It turns out to be very useful to have a borrowable version of `views::transform`. Indeed, P2728R6 even adds a dedicated new range adaptor (`views::project`) which is simply a version of `views::transform` that can be borrowed (because its callable must be a constant).

But rather than add a dedicated view for this specific case, which requires a new name but really only helps `views::transform`, we can generalize `views::transform` to address the use-case in a way that would also help all the other range adaptors that take callables. At the very least, in `views::transform(r, f)` if `r` is borrowed and `f` is empty, an implementation can simply put `f` in the

`transform_view<R, F>::iterator` directly (rather than a `transform_view<R, F> *`) which would allow it to be borrowed. The same could be said for other range adaptors that take callables as well, which seems like a more useful approach as well as not requiring new names for every adaptor.

The main question then is what the criteria should be for when `transform_view<R, F>` should be a borrowed range (when `R` is):

- `is_empty_v<F>` (`range-v3` already does this - not for conditionally borrowed, but just to decide whether to store the callable by value in the iterator)
- `sizeof(F) <= sizeof(void *)` and `is_trivially_copyable_v<F>` (this means that when transforming with a function pointer, the function pointer itself can live in the iterator - which takes the same amount of space as the parent pointer, except with one less indirection)
- something else?

This question is a little simpler for `views::transform` (which only needs to potentially store `f` in the adapted iterator) than it is for `views::filter` (which would need not only the predicate but also the underlying sentinel, so this may not be worthwhile). This would need to be carefully considered.

§ 3 Why care about borrowed_range?

`borrowed_range` is important because it allows the standard range adaptors and views to know that a particular view is a proper view – that is, a reference type, not an expensive-to-copy type like a `std::vector<int>`. Certain views refuse to work with non-borrowed_range ranges, because the result might dangle. For instance, you cannot construct a subrange from a non-borrowed_range. In other cases, an rvalue range might be wrapped in an `owning_view` to turn it into a view, if it was not already a borrowed_range. So `std::vector<int>({1,2,3,4,5,6,7,8,9}) | take(3)` would take the rvalue vector, wrap it in an `owning_view`, and the result is a non-borrowed_range that is move-only.

Here is the part of `[range.range]` that describes `borrowed_range`:

```
template<class T>
concept borrowed_range =
    range<T> && (is_lvalue_reference_v<T> || enable_borrowed_range<remove_cvref_t<T>>);
```

- 4 Let `U` be `remove_reference_t<T>` if `T` is an rvalue reference type, and `T` otherwise. Given a variable `u` of type `U`, `T` models `borrowed_range` only if the validity of iterators obtained from `u` is not tied to the lifetime of that variable.
- 5 [Note 2: Since the validity of iterators is not tied to the lifetime of a variable whose type models `borrowed_range`, a function with a parameter of such a type can return iterators obtained from it without danger of dangling. ? end note]

To increase the interoperability of the templates in `std::ranges` and to avoid unnecessary wrapping, we should make as many views borrowed_ranges as is reasonable. It is probably unreasonable to do so if there is a significant cost involved. To make a view a borrowed_range means copying its state out of the view itself, and into the iterator instead.

Note that copying state to the iterator is not sufficient to guarantee borrowed-ness, though is it necessary. Most views take an existing view `v` and do something with it, making a new view over the existing one. For instance, `transform_view<V, F>` creates a view by transforming the elements of `v` using the invocable `F`.

A particular view specialization is only borrowed if it satisfies the `borrowed_range` concept, *and* `v` does as well. This is important, because if a view is not borrowed, no view that uses it will be either. In `rng | v1 | v2 | ... | vn`, if *any* `vi` is not borrowed, the entire resulting view is not.

§ 4 Status quo

Currently, there are several views that are never borrowed_ranges that potentially could be. They are: `transform_view`, `zip_transform_view`, `adjacent_transform_view`, `filter_view`, `take_while_view`, `chunk_by_view`, `join_view`, `join_with_view`, `split_view`, `lazy_split_view`, and `cartesian_product_view`.

The status quo is also not as memory safe as it could be. Many of the views above have iterators that refer back to the view to get the end of the underlying sequence `v.base_`. Consider this example.

```
auto v = /* some view containing 100 elements */;
int count = 0;
auto const last = std::ranges::end(v);
for (auto it = std::ranges::begin(v); it != last; ++it) {
    if (++count == 50) {
        v = /* new stuff, where end(v) is different */;
        // Some subsequent operations on 'it' will reach back into 'v' for some if
```

```

    // 'it' needs to know the end of the v.base_. Even though we made a copy of
    // ranges::end(v) -- 'last' -- we are not protected.
}
}

```

This is unusual code to be sure, but nothing in the standard indicates that this is UB. Rather than just update the wording with preconditions that indicate the UB, we would like to copy `base_.end()` into the iterators of all the views above. This is a necessary step in making a view borrowable, but is probably a good idea, independent of borrowability concerns.

§ 4.1 The easy ones

`join_view` is easiest of all. The only modifications it requires are to copy the view's `base_.end()` into the iterator, and specialize `enable_borrowed_range`. `join_view`'s borrowability is predicated on its being a `forward_range`. That's because `join_view` and `join_with_view` have these caches:

```

non-propagating-cache<iterator_t<V>> outer_;           // exposition only, present only
                                                         // when !forward_range<V>
non-propagating-cache<remove_cv_t<InnerRng>> inner_;    // exposition only, present only
                                                         // if is_reference_v<InnerRng> is false

```

If a `join_view` uses either of its two caches, it will be an `input_range`. So this will do the trick:

```

template<typename T>
constexpr bool enable_borrowed_range<join_view<T>> =
    enable_borrowed_range<T> && forward_range<join_view<T>>;

```

Four more views are easy to change with little impact: `transform_view`, `zip_transform_view`, `adjacent_transform_view`, and `take_while_view`. Each of these views stores its invocable in the view, and has an iterator type that contains a pointer back to the view. However, as stated in the Motivation section, if the invocable (“F f” for the first three, “Pred pred” for `take_while_view`, just “f” hereafter) has a size no larger than a pointer and is trivially copyable, we could just replace the iterator's (or sentinel's, in the case of `take_while_view`) back-pointer to the view with a copy a f. This would enable borrowed-ness, and also would make these views slightly more efficient by removing an indirection. We could have insisted that F be empty, and most Fs that people use with these views will be. However, since we're already replacing a pointer, we might as well allow for a small amount of state in F. Note that since the change to the iterator/sentinel data members depends on `sizeof(F)`, the change is statically conditional.

§ 4.2 The low-cost ones

`filter_view` could get the same treatment as the easy ones above (the ones that require modification), but its iterator also needs to know where the end of the underlying view is, so the sentinel would also need to be added to the iterator. This would increase the size of the iterator by at most the size of two pointers.

`chunk_by_view` is mostly the same story as `filter_view`. However, for bidirectional specializations of the view, the iterator's `operator--` would also need the beginning of the underlying view v. This would increase the size of the iterator by at most the size of two or three pointers for forward and bidirectional views, respectively.

`split_view` and `join_with_view` each take a view V and a view Pattern to use to do the join or split, respectively. Clearly, to be borrowable V must be borrowable. If Pattern is borrowable as well, then the entire `split_view/join_with_view` is too. However, even if Pattern is not borrowable, if we could copy Pattern into the iterator, that would work just as well. A Pattern that is trivially copyable and no larger than `sizeof(Pattern) <= sizeof(void*) * 2` is acceptable to copy, since a borrowed_range Pattern will typically be about the size of two pointers.

`lazy_split_view` is nearly the same story as `split_view`, except that it has an additional cache:

```

non-propagating-cache<iterator_t<V>> current_;           // exposition only, present only
                                                         // if forward_range<V> is false

```

As with `join_view`, the cache is in use if `forward_range<lazy_split_view<V, Pattern>>` is false. This means its borrowability depends on V, Pattern, and `forward_range<lazy_split_view<V, Pattern>>`.

§ 4.3 The other one

`cartesian_product_view` has an unbounded number of views it may be specialized with, so it's probably a poor candidate for copying stuff into the iterator.

§ 5 Proposed changes

All these changes are in `std::ranges`.

§ 5.1 join_view

Change `join_view::iterator` to store the `join_view`'s `base_.end()` instead of a pointer to the `join_view`. Then, add this specialization:

```
template<typename T>
constexpr bool enable_borrowed_range<join_view<T>> =
    enable_borrowed_range<T> && forward_range<join_view<T>>;
```

§ 5.2 Create implementation-only traits

```
template<typename F>
constexpr bool tidy-func = is_trivially_copyable_v<F> && sizeof(F) <= sizeof(void*);
template<typename V>
constexpr bool tidy-view = is_trivially_copyable_v<V> && sizeof(V) <= sizeof(void*) * 2;
```

`tidy-func` evaluates to true when an invocable is a candidate for copying into a view's iterators. `tidy-view` evaluates to true when a view is a candidate for copying into a view's iterators.

§ 5.3 The easy ones

`transform_view`, `zip_transform_view`, `adjacent_transform_view`, and `take_while_view` all have the same kind of changes, except that `take_while_view`'s sentinel is changed, not its iterator. Using `transform_view` as an example:

In 26.7.9.3 [range.transform.iterator]:

```
namespace std::ranges {
    template<input_range V, move_constructible F>
        requires view<V> && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<V>> &&
            can_reference_invoke_result_t<F&, range_reference_t<V>>>
        template<bool Const>
        class transform_view<V, F>::iterator {
        private:
            using Parent = maybe-const<Const, transform_view>;           // exposition only
            using Base = maybe-const<Const, V>;                         // exposition only
            iterator_t<Base> current_ = iterator_t<Base>();             // exposition only
-           Parent* parent_ = nullptr;                                  // exposition only
+
+           using f-access =
+               conditional_t<tidy-func<F>, movable-box<F>, Parent*>;   // exposition only
+
+           [[no_unique_address]] mutable f-access f_access_;          // exposition only
+
+           constexpr maybe-const<Const, F>& f() const                  // exposition only
+           {
+               if constexpr (tidy-func<F>)
+                   return *f_access_;
+               else
+                   return *f_access_->fun_;
+           }
+ }
```

Then, change every place that previously used `parent_->fun_` to use `f()` instead. Then, replace the initialization of `parent_` in constructors with the conditional initialization of `f_access_` with `parent.fun_` if `tidy-func<F>` is true, and `addressof(parent)` otherwise.

Finally, add the `enable_borrowed_range` specialization:

```
template<typename V, typename F>
constexpr bool enable_borrowed_range<transform_view<V, F>> = enable_borrowed_range<V> && tidy-func<F>;
```

`mutable` is used on `f_access_` to make sure that the reference to `f()` is non-const. This gives the same behavior as the status quo – even when using `f()` in a const member function of iterator, like `operator*`, we still call `f()` as if we were doing so through a pointer back to the view.

§ 5.4 filter_view

`filter_view::iterator` gets a similar change as the one for `transform_view::iterator`, slightly modified. The difference is that the nested type, data member, and member function look like this:

```
using pred-access =
    conditional_t<tidy-func<Pred>, movable-box<Pred>, filter_view*>;
```

```
[[no_unique_address]] pred_access pred_access_;

constexpr Pred& pred()
{
    if constexpr (tidy_func<Pred>)
        return *pred_access_;
    else
        return *pred_access_->pred_;
}
```

Note the absence of *maybe-const*, or *pred()* *const*. `filter_view::iterator` has no *const* variant, and it uses its predicate exclusively within non-*const* member functions.

Additionally, `filter_view::iterator` has `base_.end()` from the `filter_view` stored in it. Again, the `base_` iterator copy happens unconditionally. `filter_view` gets an `enable_borrowed_range` specialization as well:

```
template<typename V, typename Pred>
constexpr bool enable_borrowed_range<filter_view<V, Pred>> =
    enable_borrowed_range<V> && tidy_func<Pred>;
```

§ 5.5 chunk_by_view

`chunk_by_view::iterator` gets the same changes as `filter_view::iterator`, plus it also has `base_.begin()` from the `chunk_by_view` stored in it. Again, this happens unconditionally. `chunk_by_view` gets a `enable_borrowed_range` specialization as well.

§ 5.6 lazy_split_view, split_view, and join_with_view

These three are pretty similar to each other. Instead of the iterator conditionally storing a copy of an invocable, each conditionally stores a copy of the view's *Pattern*:

```
constexpr bool store_pattern = see below; // exposition only

using pattern_access =
    conditional_t<store_pattern, Pattern, Parent*>; // exposition only

// mutable is only used for join_with_view.
[[no_unique_address]] mutable pattern_access pattern_access_; // exposition only

// maybe-const is used in join_with_view only; the others return const Pattern &.
constexpr maybe_const<Const, Pattern>& pattern() const // exposition only
{
    if constexpr (store_pattern)
        return pattern_access_;
    else
        return pattern_access_->pattern_;
}
```

`store_pattern` is true if the iterator is a `forward_iterator` and `tidy-view<Pattern>` is true; it is false otherwise.

The *Pattern* is never stored in the iterator when the whole view is an input range, since the cache in the view must be used regardless; this makes the view non-borrowable. `split_view` does not have such a cache, so it doesn't use the `forward_range` part of the condition. Also, only `join_with_view` needs to use *maybe-const* for the return type of `pattern()`; the other two just return `const Pattern &`. This is because `join_with_view` produces its pattern as part of its output; the other two only read the values out of the pattern, so it makes no difference if the pattern is *const* or not.

Otherwise, the changes are the same as before – conditionally copy *Pattern* into the iterator, and unconditionally copy the view's `base_.end()` into the iterator.

All three views get `enable_borrowed_range` specialization like this, except that the `split_view` one has no `forward_range` requirement:

```
template<typename V, typename Pattern>
constexpr bool enable_borrowed_range<lazy_split_view<V, Pattern>> =
    enable_borrowed_range<V> && (enable_borrowed_range<Pattern> || tidy-view<Pattern>) &&
    forward_range<lazy_split_view<V, Pattern>>;
```

§ 6 Implementation experience

One of the authors implemented the changes suggested above, for all the views discussed. The implementation was done by taking the `libstdc++` implementations, copying them into a new header, in a new namespace under `std::ranges`, and altering them. The

implementation can be found [here](#). The header is accompanied by a test file, and a small perf test. The implementation was very straightforward.

§ 7 Costs versus benefits

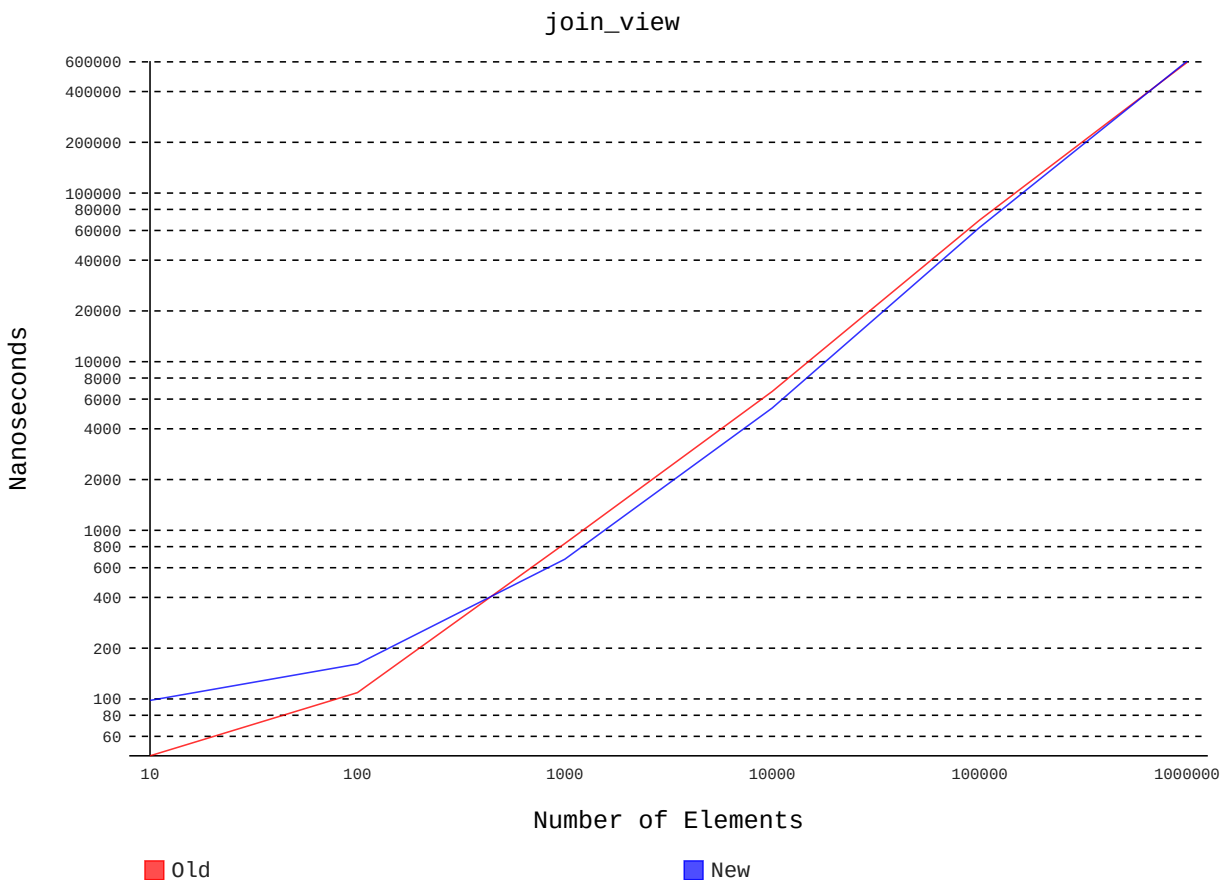
`join_view` has the simplest proposed change. It just gets a copy of the end-iterator for the underlying view `v`, and the addition of the `enable_borrowed_range` specialization.

Since it only adds an end iterator to `join_view::iterator`, it is a convenient case to test the performance impact of this change. For simple cases, like just doing a single join operation, the `join_view` changes had no performance impact that the authors could measure. However, the case is different when you pipe together several view adaptors.

For the chart and table below, “Old” indicates the status quo implementation; “New” indicates the implementation modified by the authors. This view was used:

```
auto view = subranges | views::join | views::transform(identity) | views::split(99) | views::join;
```

These are the first of the performance numbers; more follow below. All performance numbers were generated using Google Benchmark on a GCC 13 -03 build. All table numbers are in nanoseconds.



Run n	10	100	1000	10000	100000	1000000
Old	46	109	835	6664	69190	595838
New	98	161	674	5314	62558	606272

The numbers are pretty terrible at small sizes of `N`, gradually converging as `N` grows. This is likely due to the large size of `join_view::iterator` for this deeply-nested view type; for “Old”, `sizeof(view.end()) == 192`, whereas for “New”, `sizeof(view.end()) == 408`. If this view were not a `common_range`, `sizeof(view.end())` would be much smaller in both cases.

§ 7.1 The other easy ones

The other easy ones seem like clear improvements. The modifications are simple, and they’re nearly identical in performance to the status quo. In these tables and charts, “old” is the status quo implementation of the view; “new” uses a version modified to be

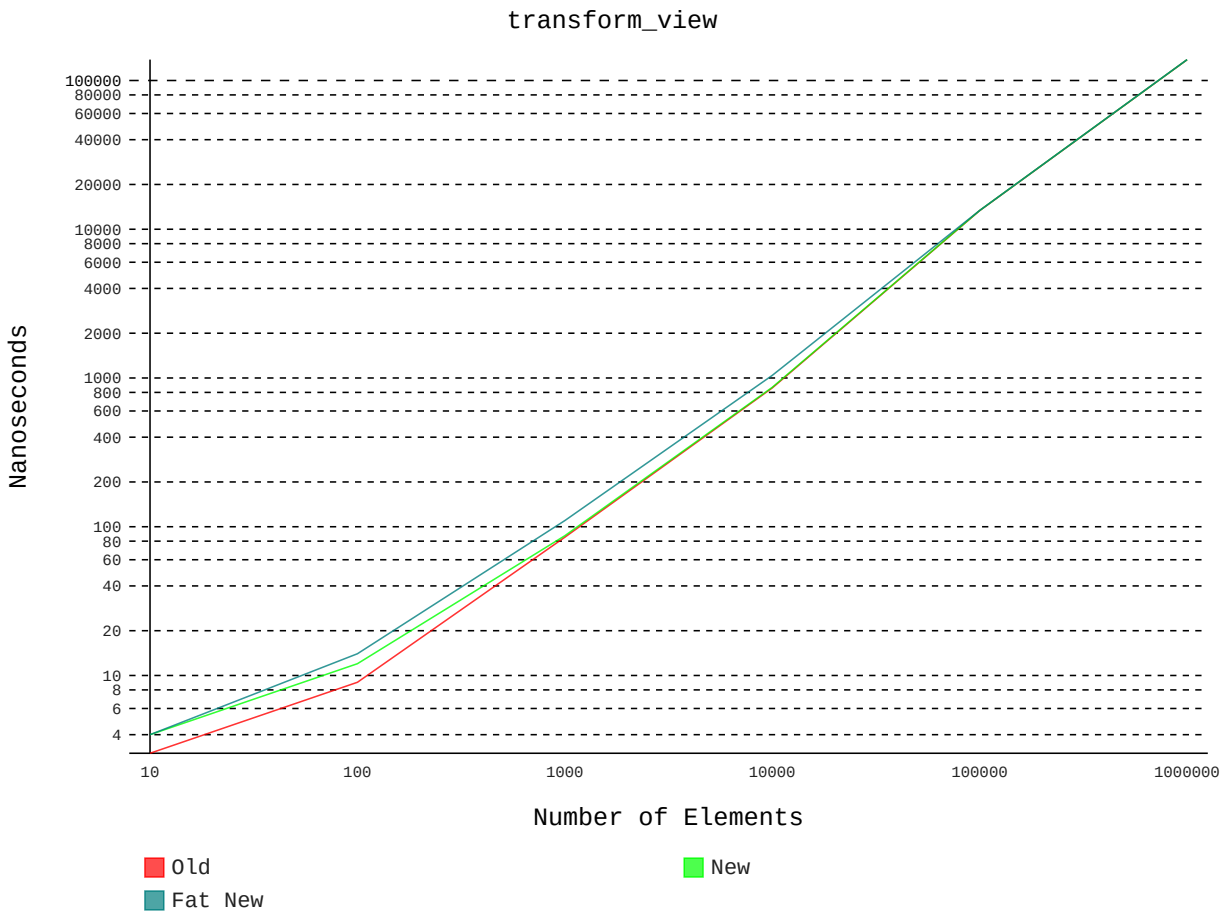
borrowable, where the invocable in use is small enough to make the view borrowable; and “fat new” is the same modified implementation as “new”, but with an invocable that *does not* allow the view to be borrowable.

Above we mentioned expectations for increased size of iterators. It turns out that in some cases, copying the invocable into the iterator actually *decreases* the size of the iterator. This is because before the change, the iterator had a pointer back to the view. If it instead contains the invocable directly, and the invocable is stored in a *movable-box* marked `[[no_unique_address]]`, the compiler can just use it without storing it in some cases.

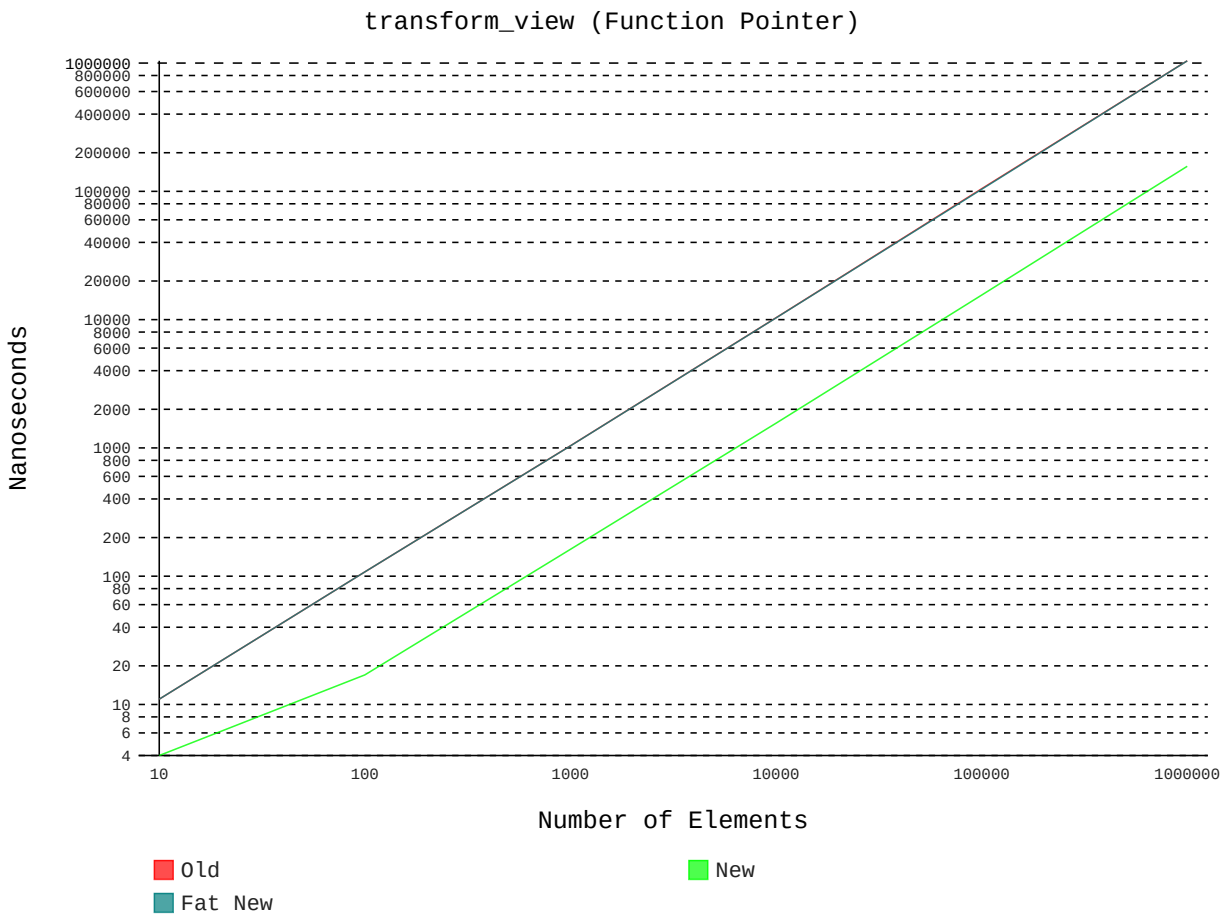
View	<code>sizeof(old.begin())</code>	<code>sizeof(new.begin())</code>	<code>sizeof(fat_new.begin())</code>
<code>transform_view</code>	16	8	16
<code>zip_transform_view</code>	24	16	24
<code>adjacent_transform_view</code>	24	16	24
<code>take_while_view</code>	8	8	8

So, there’s no size penalty for adding conditional borrowability to these four views. Here are some performance charts. After each one, we’ve listed the raw data, since the logarithmic scale makes it hard to see exact differences. Unfortunately, with a linear scale, the differences are even less comprehensible.

For each view, there are two charts. The first the result of using a very simple lambda as the callable. The second uses a pointer to a function with a body identical to the lambda.

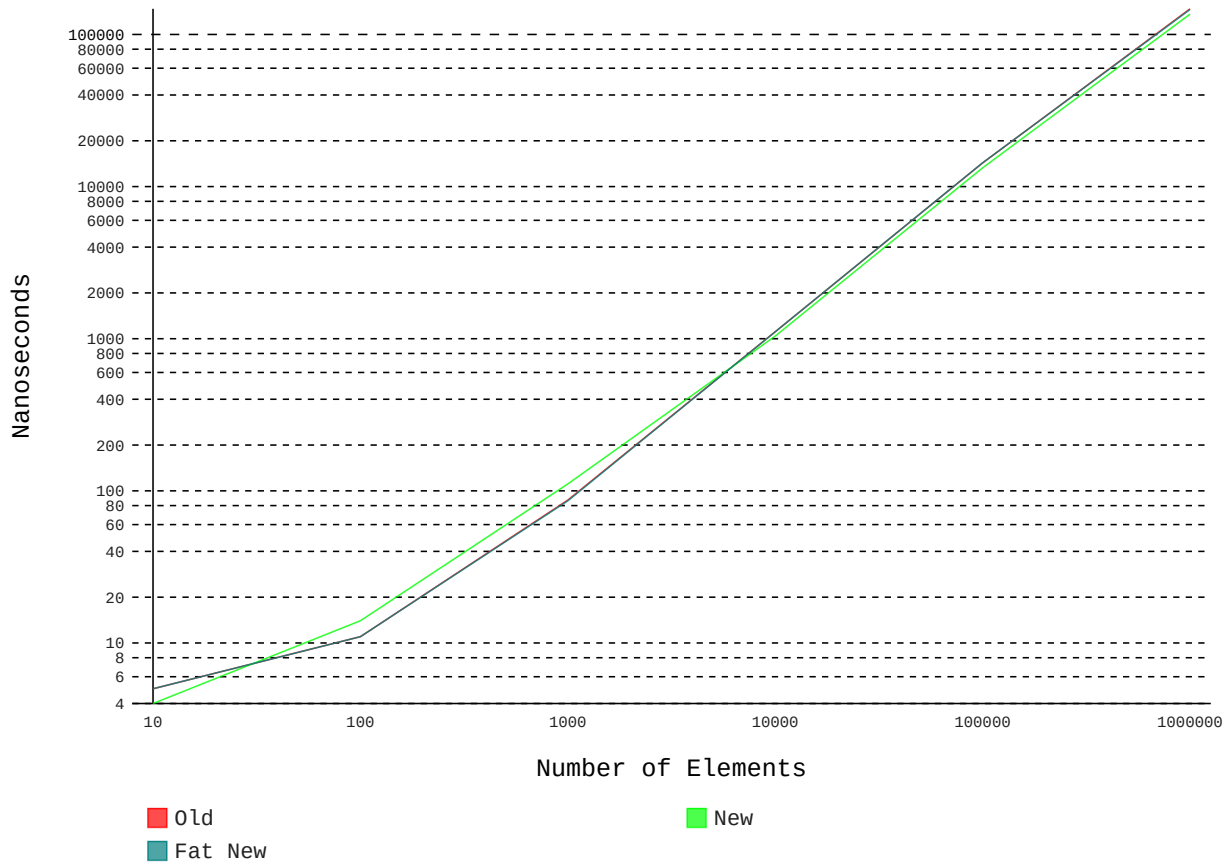


Run n	10	100	1000	10000	100000	1000000
Old	3	10	91	927	14714	157813
New	4	13	117	1115	14711	160760
Fat New	4	12	94	946	14802	157828



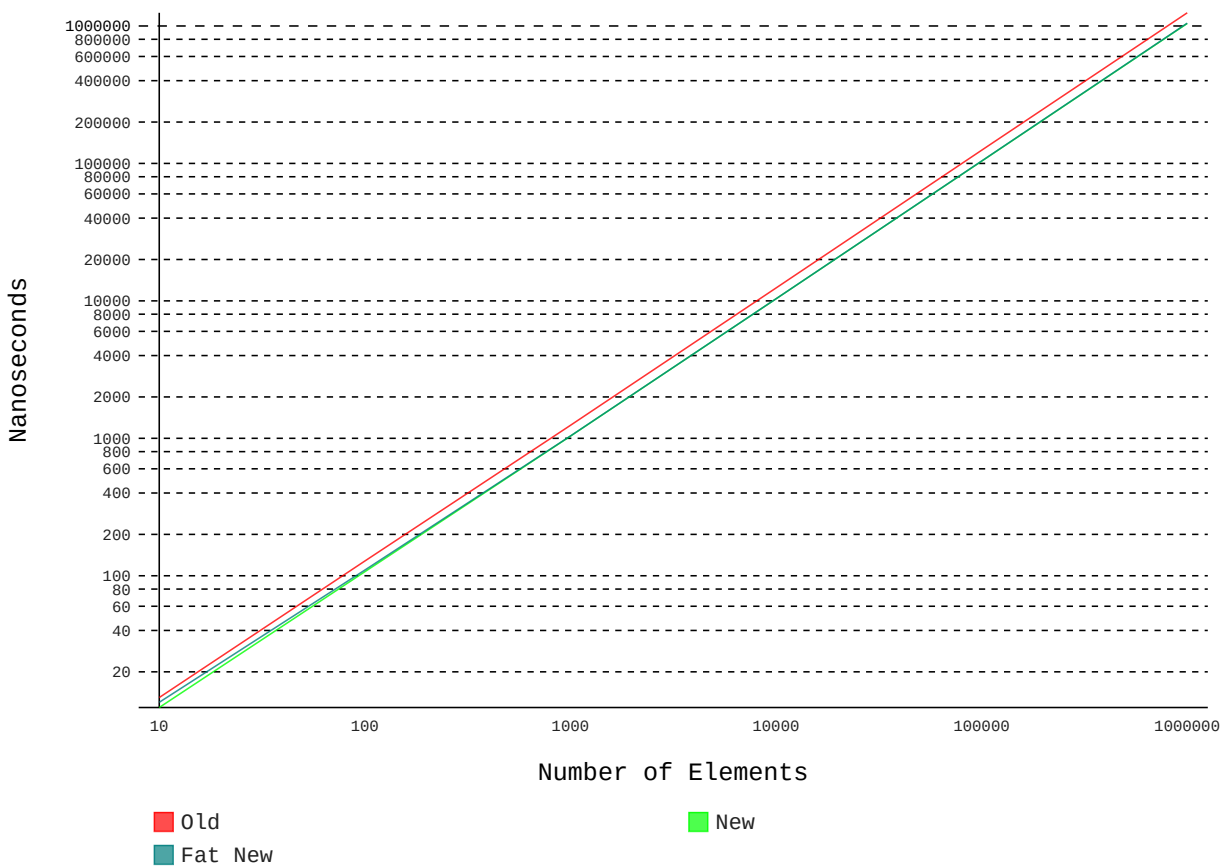
Run n	10	100	1000	10000	100000	1000000
Old	11	108	1039	10343	104773	1041585
New	4	17	162	1558	15530	156805
Fat New	11	108	1037	10314	103725	1040596

zip_transform_view



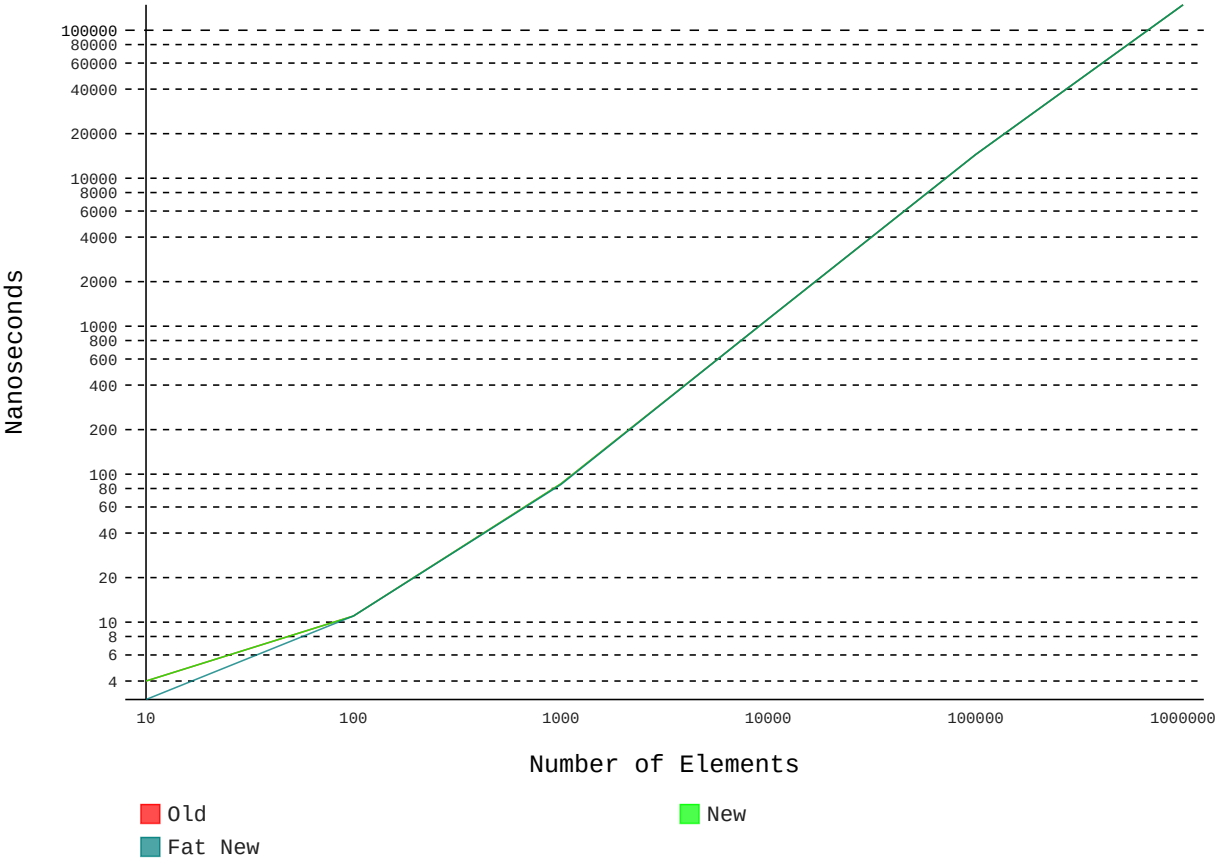
Run n	10	100	1000	10000	100000	1000000
Old	4	14	119	1219	16323	172855
New	4	15	118	1128	15212	162041
Fat New	5	11	92	1196	16083	173267

zip_transform_view (Function Pointer)

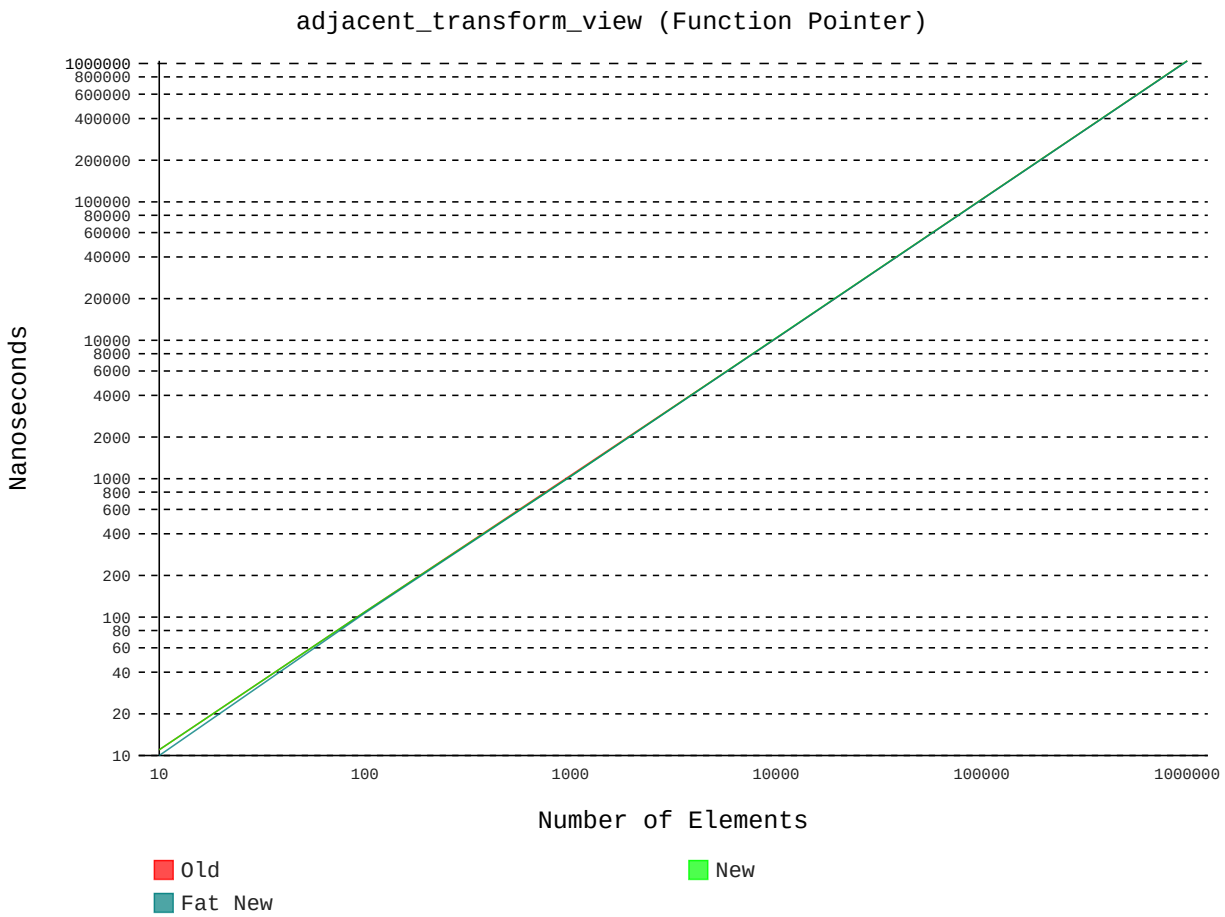


Run n	10	100	1000	10000	100000	1000000
Old	13	128	1243	12410	124728	1247042
New	11	107	1037	10346	103790	1042682
Fat New	12	110	1040	10359	104147	1048073

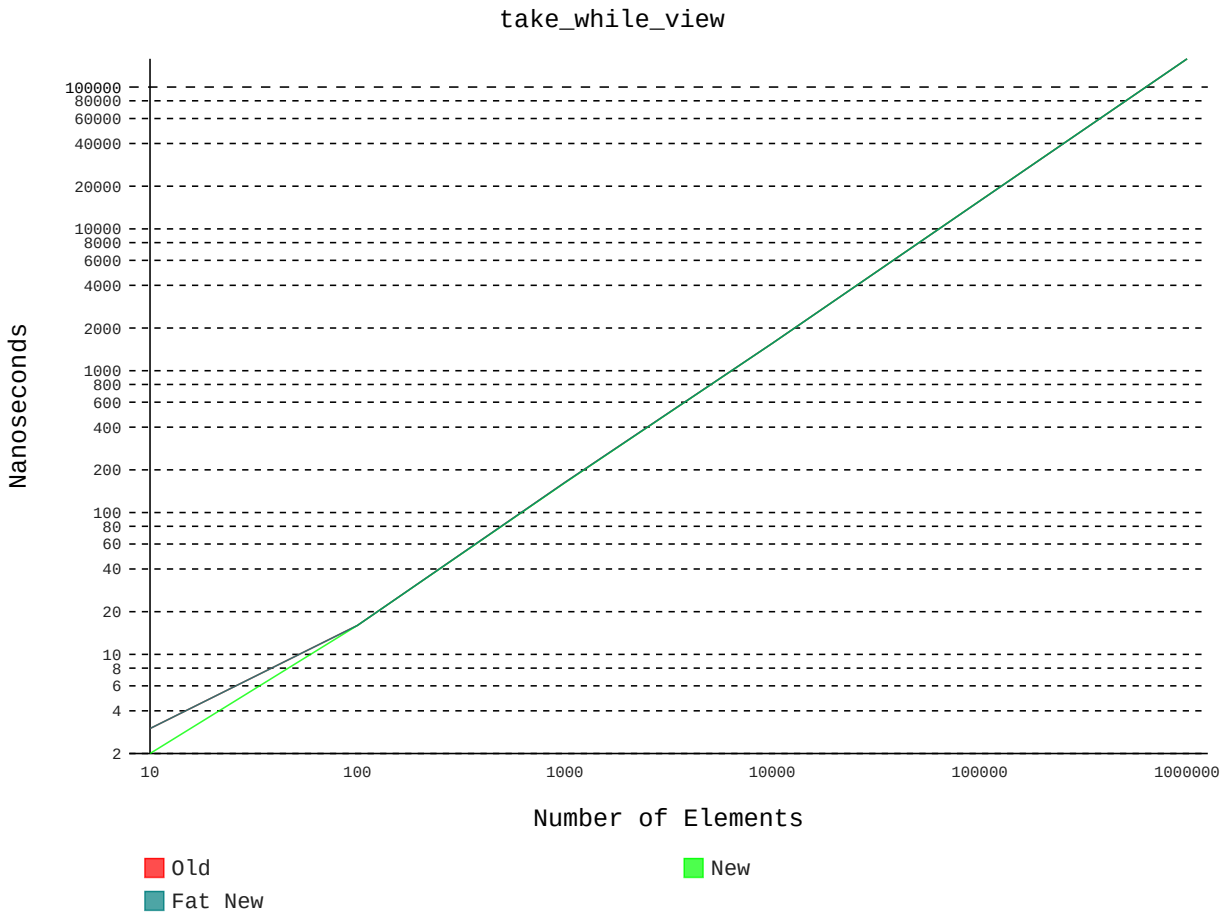
adjacent_transform_view



Run n	10	100	1000	10000	100000	1000000
Old	4	12	93	1199	16199	178819
New	4	12	92	1217	15990	171407
Fat New	4	12	93	1217	15815	172544



Run n	10	100	1000	10000	100000	1000000
Old	11	109	1054	10340	104192	1044660
New	11	109	1039	10399	104172	1045590
Fat New	10	107	1036	10321	103684	1040870



Run n	10	100	1000	10000	100000	1000000
Old	3	18	174	1711	17337	175469
New	3	18	176	1705	17180	175326
Fat New	3	18	177	1698	17295	176765

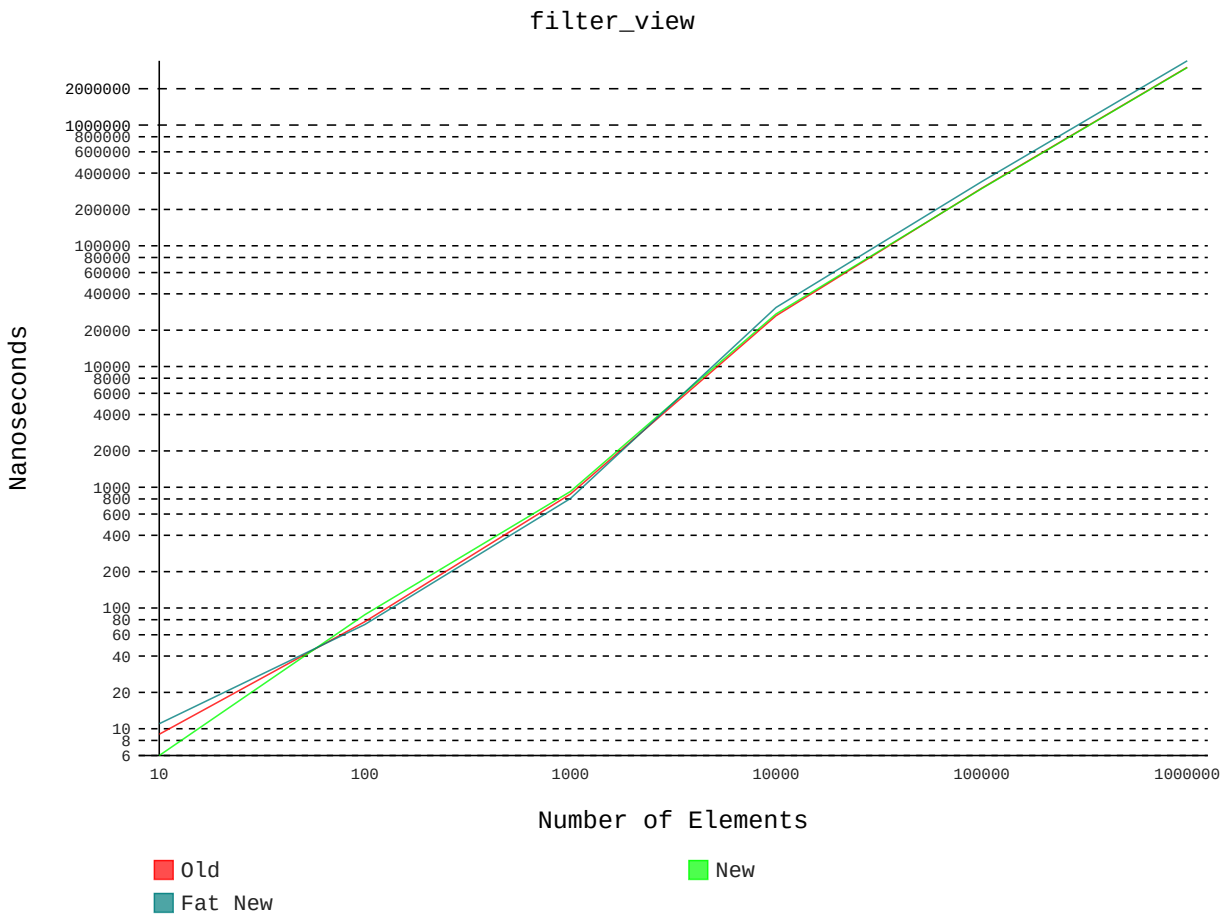
The take-away here is that changing these four views yields a negligible impact on performance for the lambda case, and modest to dramatic improvements in the function pointer case. In particular, `take_while_view` produces performance numbers that are so similar to the status quo that the lines completely overlap. Where the numbers differ only slightly, the new implementations are slightly better for most sizes of input.

§ 7.2 The low-cost ones

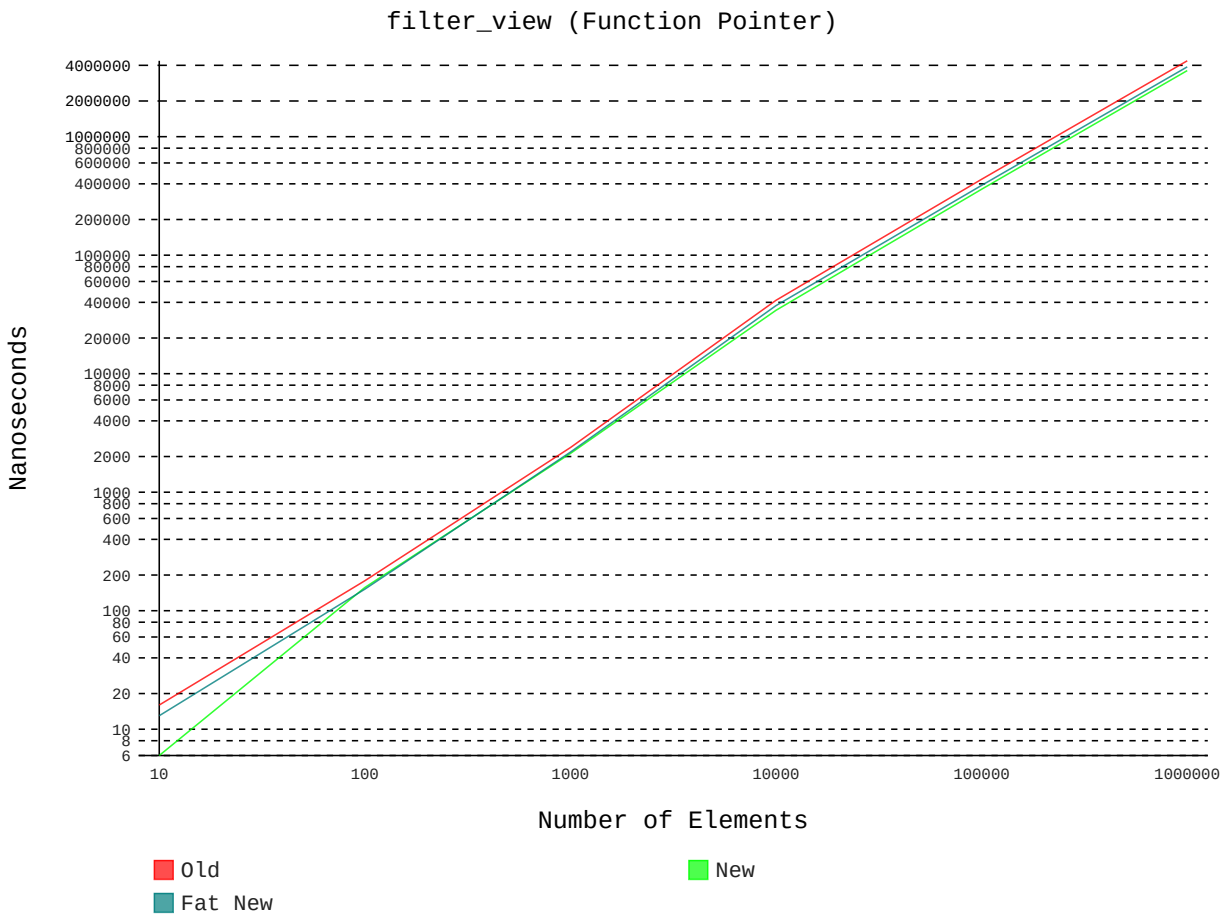
The low-cost ones are more of a mixed bag than the easy ones. Let's first look at `filter_view` and `chunk_by_view`. We'll look at the others together, since they are more like each other than these two.

Let's look at sizes. As before, the size penalty is sometimes less, when the invocable is small. Unlike before, there is a size penalty here when the invocable does not fit. That's mainly because we also copied `base_.end()` (from the view's `v base_`) into the iterator.

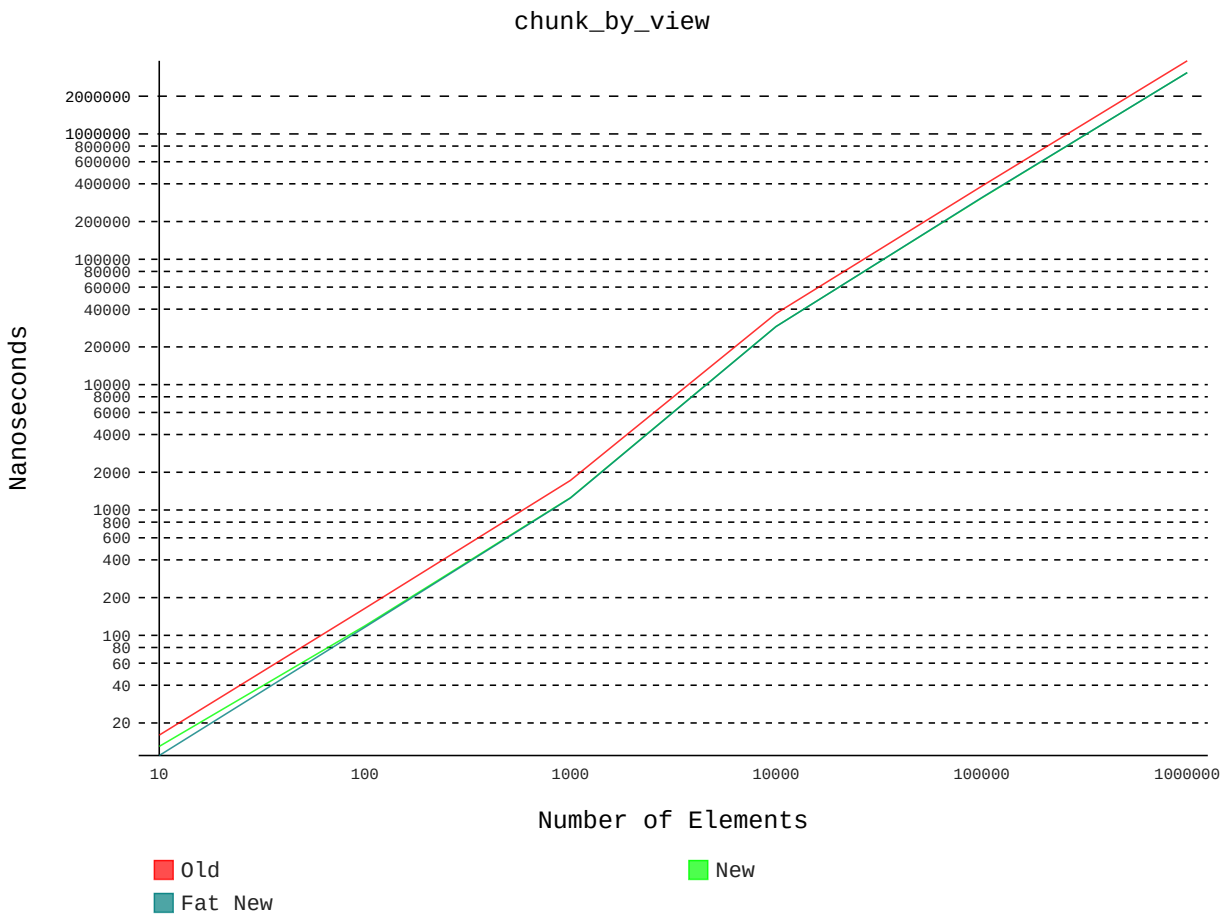
View	<code>sizeof(old.begin())</code>	<code>sizeof(new.begin())</code>	<code>sizeof(fat_new.begin())</code>
<code>filter_view</code>	16	16	24
<code>chunk_by_view</code>	24	32	40



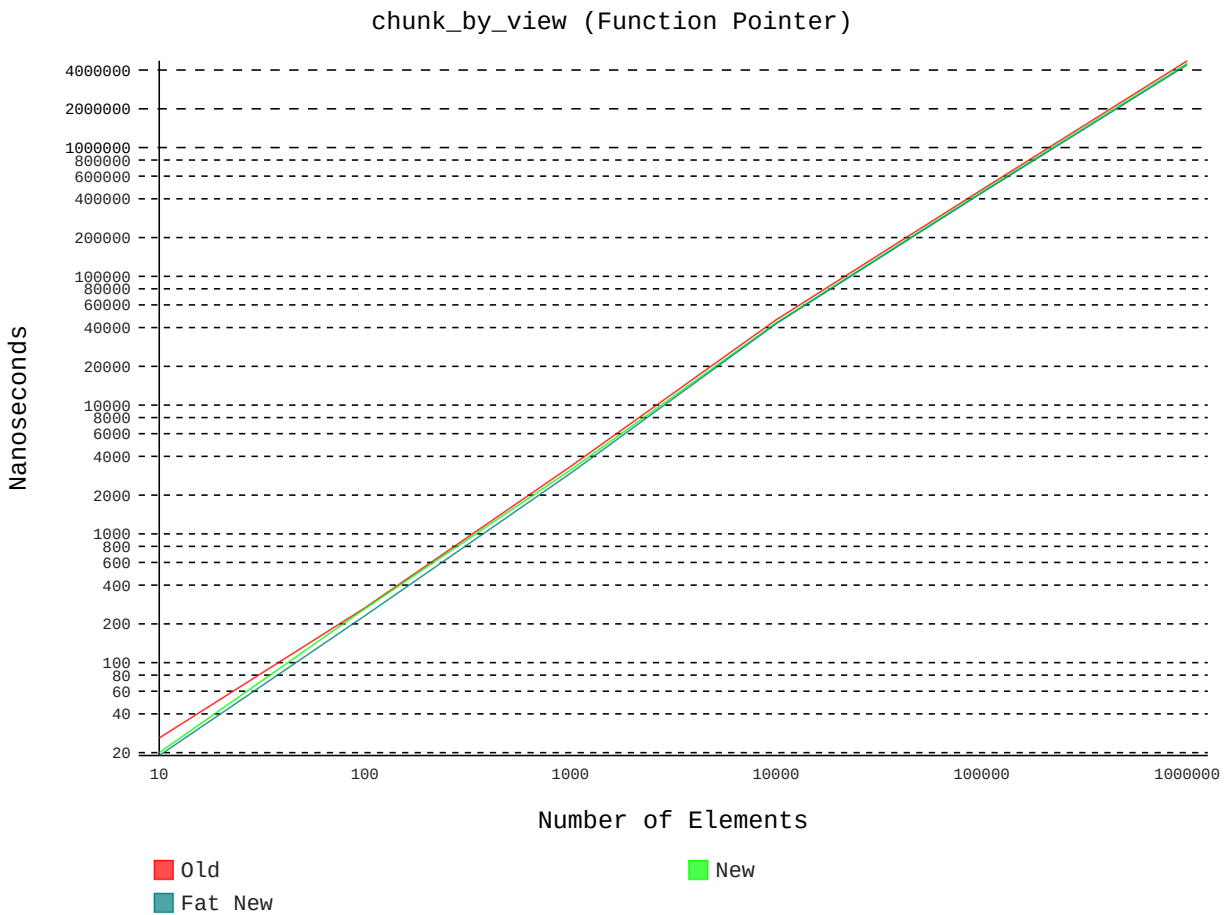
Run n	10	100	1000	10000	100000	1000000
Old	9	93	988	28475	324653	3280019
New	11	104	1386	34734	386109	3865245
Fat New	12	111	1099	29161	327035	3335268



Run n	10	100	1000	10000	100000	1000000
Old	16	179	2381	41636	437130	4367355
New	6	157	2124	34282	360168	3611216
Fat New	13	152	2179	37559	388305	3883130



Run n	10	100	1000	10000	100000	1000000
Old	23	216	2376	42229	438928	4424799
New	16	150	1528	33707	367695	3729119
Fat New	16	160	1526	35283	365044	3703705



Run n	10	100	1000	10000	100000	1000000
Old	26	266	3331	45865	469879	4718487
New	20	262	3132	43524	452269	4514382
Fat New	19	231	2952	42788	443756	4395832

As you can see, there is a small but significant performance penalty for `filter_view`. For `chunk_view`, however, making it borrowable is a significant perf boost, even if the invocable is too large to get stashed in the iterator. This means that simply having `base_.end()` in the iterator is beneficial. Who knew?

§ 7.2.1 The multi-view low-cost views

These views are together because they are similar to each other in the data they operate on. Each one takes a view `v` like any other view, but also another view `Pattern`, that is used to do the split or join. The range adaptor for each of these views also accepts a single element instead of the pattern, from which it will construct a `single_view` from that element.

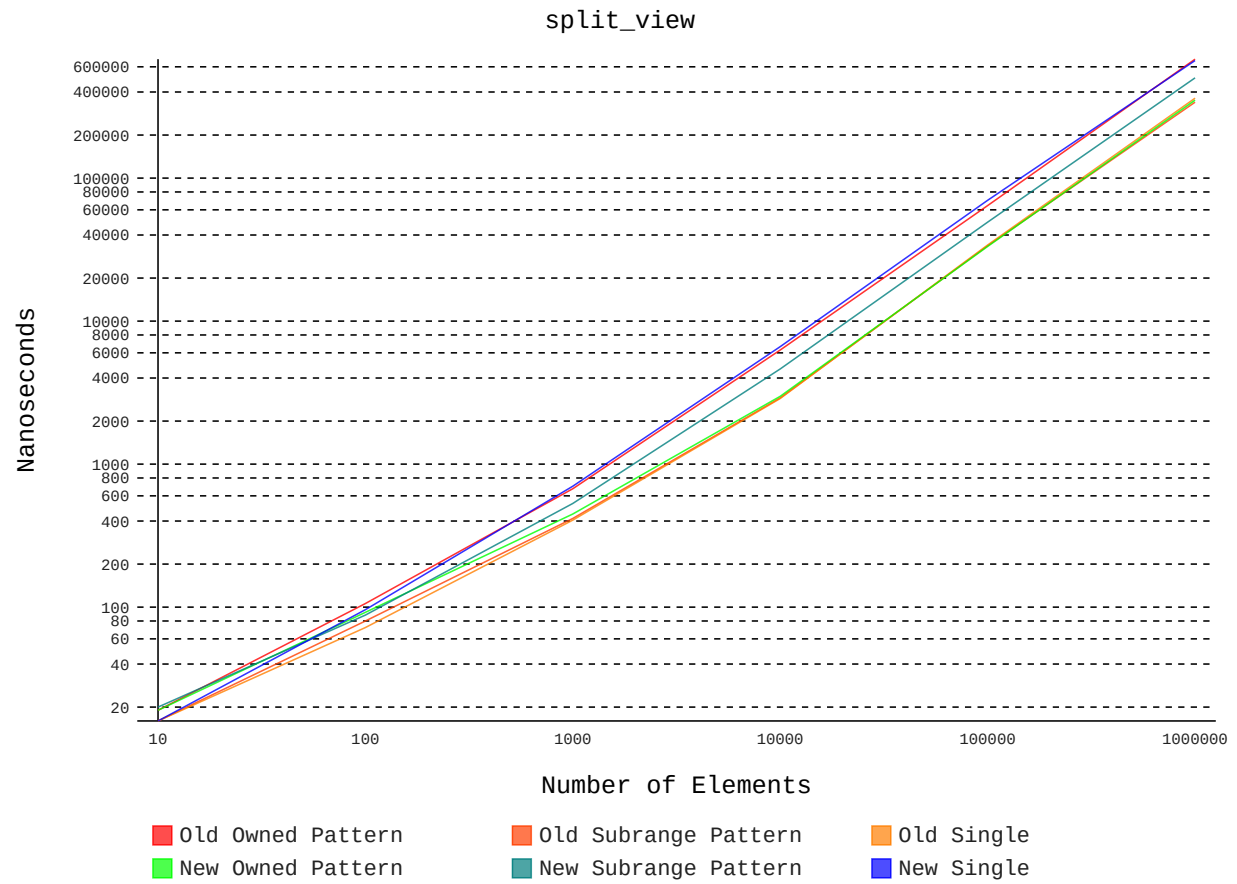
In the tables below, “old” is an iterator from the status quo implementation; “new” is an iterator from a view with a `ranges::subrange` given for `Pattern`; “new single” is an iterator from a view with a single element (in the form of a `single_view` given for `Pattern`; and “fat new” is an iterator from a view with a non-trivially-copyable `Pattern` (an `owning_view` was used).

View	<code>sizeof(old.begin())</code>	<code>sizeof(new.begin())</code>	<code>sizeof(new_single.begin())</code>	<code>sizeof(fat_new.begin())</code>
<code>lazy_split_view</code>	24	40	32	32
<code>split_view</code>	40	56	48	48
<code>join_with_view</code>	32	48	40	40

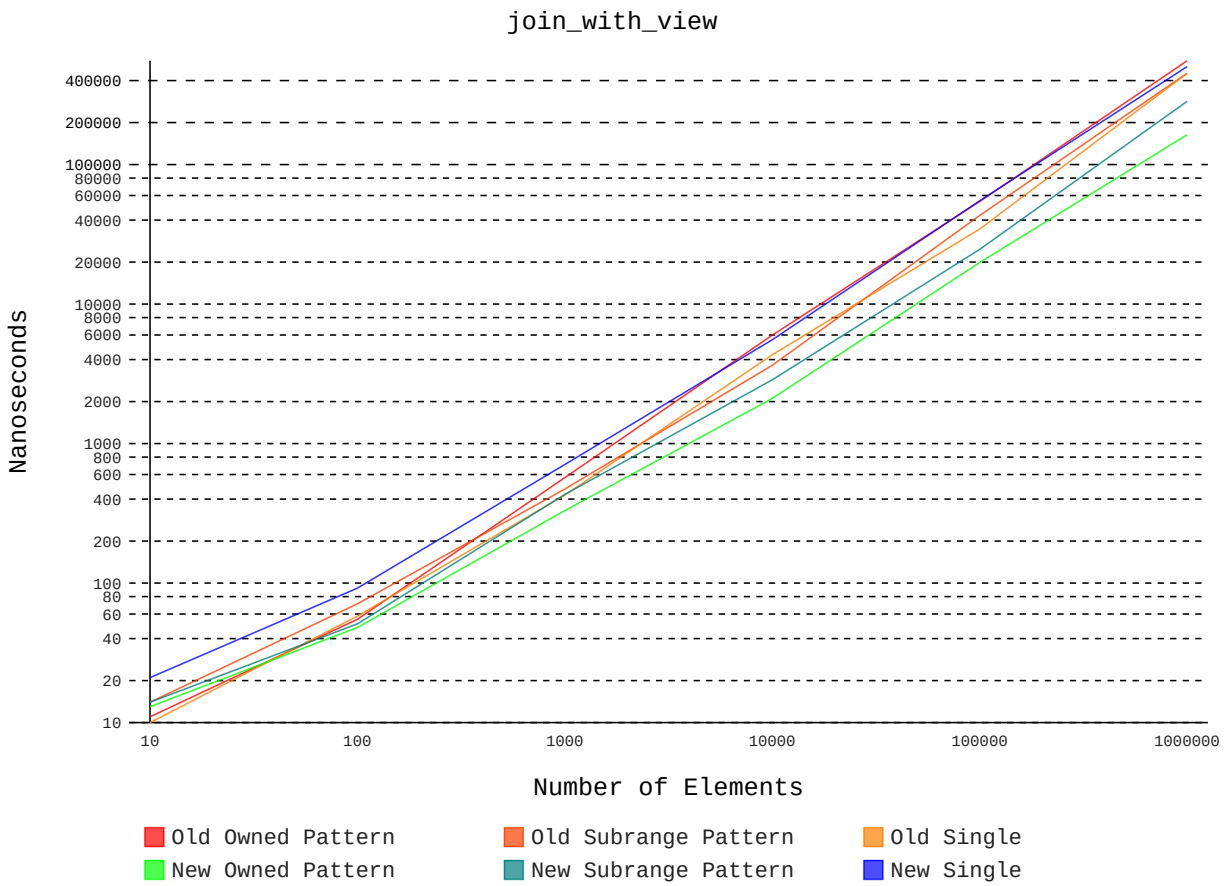
There is definitely a size penalty for modifying these views to be borrowable. `new` is always a single pointer larger than `fat_new`, because `fat_new` uses a pointer to the view when the pattern is not copied into the iterator. `fat_new` is therefore basically like `old`, except that it has `base_.end()` copied into it as well. `new_single` can be smaller than `new` when the `single_view` is small, such as in `v | ranges::split_view('-')`. In that case, storage of the `single_view` actually gets optimized away by the compiler, like we saw with simple invocables previously.

Now, the performance numbers. In these charts and tables, “old” and “new” still refer to the status quo and conditionally borrowed implementations, respectively. “Owned” indicates that an `owning_view` was used for `Pattern`; “subrange” indicates that a

ranges::subrange was used for Pattern; and “single” indicates that a single_view (always <= sizeof(void*)) was used for Pattern.

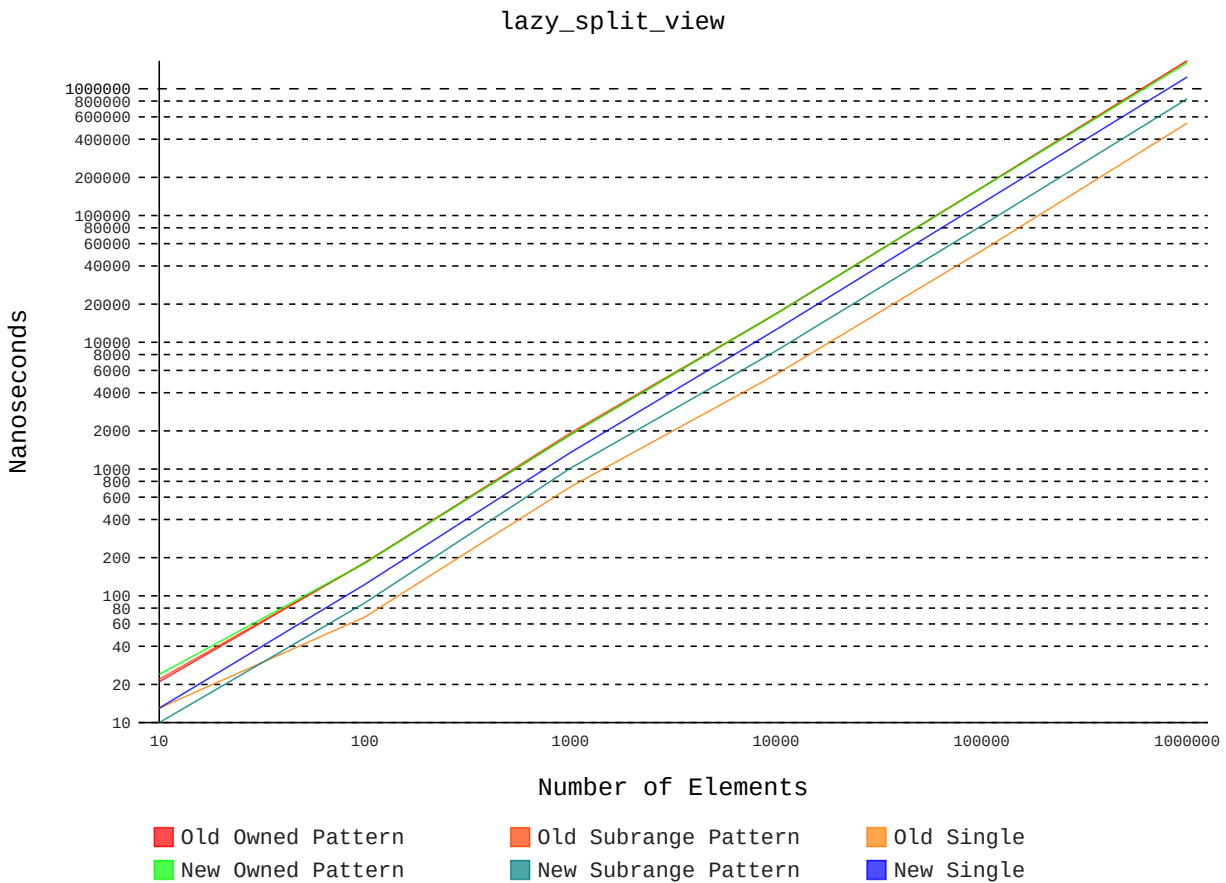


Run n	10	100	1000	10000	100000	1000000
Old Owned Pattern	21	89	462	3211	36950	388447
Old Subrange Pattern	19	84	404	3315	36488	394279
Old Single	15	91	485	3301	37417	407012
New Owned Pattern	20	99	444	3354	36560	391778
New Subrange Pattern	20	90	443	3345	36013	404278
New Single	19	73	414	3353	36350	383636



Run n	10	100	1000	10000	100000	1000000
Old Owned Pattern	13	80	539	6177	59575	625030
Old Subrange Pattern	13	68	685	6634	52002	548042
Old Single	12	63	531	4633	52861	578949
New Owned Pattern	11	44	321	1899	19805	233189
New Subrange Pattern	13	55	320	2022	21655	234532
New Single	79	96	761	7374	70770	620282

split_view and join_with_view both show mixed results, but tend to do better on larger values of n. For both of these views, you have options that are better or worse than the status quo's equivalent. So, by profiling and changing the range given for Pattern accordingly, users can meet or beat the previous performance numbers.



Run n	10	100	1000	10000	100000	1000000
Old Owned Pattern	25	218	2472	22743	224713	2302364
Old Subrange Pattern	26	218	2248	20687	204262	2062595
Old Single	13	84	793	5979	56787	650747
New Owned Pattern	21	176	1963	16970	164141	1669101
New Subrange Pattern	11	95	1090	8871	87545	895048
New Single	13	98	1312	12310	118646	1234266

The results for `lazy_split_view` is disappointing, in that the best results are available only from one of the “Old” runs.

For all three views, it is surprising that “New Single” performs so poorly. If the implementation is changed so that `Pattern` is never copied into the iterator, “New Single” performs about as well as the other “New” runs. So, the problem really is the storage of the `single_view`, and not the storage of `base_.end()`. So far, it is a very mysterious result.

§ 7.3 Costs/benefits conclusion

The size penalty is low-to-none (and occasionally negative) for all of these views’ iterators. Further, the correlation of iterator size to performance seems to be very low. We feel that we can safely ignore the size penalty.

The perf numbers above come from modifying only one implementation. However, they are promising in that, except for `lazy_split_view`, there were no definite performance penalties. All other perf regressions were for a certain range of `N`, a certain size of callable or `Pattern`, etc. In other words, the user has a means of meeting or exceeding the perf results of the status quo, by doing some profiling and some code tweaks.

The fact that the results were mixed indicates that the perf differences were probably largely specific to the testing methodology, specifics of `libstdc++`, etc. Similarly mixed results will probably be found for other implementations as well.

The two notable exceptions are `lazy_split_view`, whose status quo `single_view` run is better than all the modified runs, and `chunk_by_view`, which is way better with the proposed changes.

§ 8 References

[P2017R1] Barry Revzin. 2020-02-19. Conditionally borrowed ranges.

<https://wg21.link/p2017r1>